excercise: take an interger array of 104, 10**5, 10**6 elements. try accessing the array elements using np 1,2,4 and tell me the run total run_time for this accessing elements.**

```c
#include <stdio.h>

#include <stdlib.h>

#include <mpi.h>


void test_array(long array_size, int rank, int size) {


   int *arr = (int *)malloc(array_size * sizeof(int));


   for (long i = 0; i < array_size; i++) {

      arr[i] = i;

   }


   long chunk = array_size / size;

   long remainder = array_size % size;


   long start, end;


   if (rank < remainder) {

      start = rank * (chunk + 1);

      end = start + chunk;

   } else {

      start = rank * chunk + remainder;

      end = start + chunk - 1;

   }
```

```
    printf("Rank %d -> start: %ld end: %ld\n", rank, start, end);


    MPI_Barrier(MPI_COMM_WORLD);

    double start_time = MPI_Wtime();


    volatile int temp;

    for (long i = start; i <= end; i++) {

        temp = arr[i];

    }


    double local_time = MPI_Wtime() - start_time;


    double max_time;

    MPI_Reduce(&local_time, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);


    if (rank == 0) {

        printf("Array Size: %ld | Processes: %d | Execution Time: %f seconds\n\n",

               array_size, size, max_time);

    }


    free(arr);

}


int main(int argc, char *argv[]) {


    MPI_Init(&argc, &argv);
```

```c
    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Comm_size(MPI_COMM_WORLD, &size);


    long sizes[] = {10000, 100000, 1000000};


    for (int i = 0; i < 3; i++) {

        test_array(sizes[i], rank, size);

        MPI_Barrier(MPI_COMM_WORLD);

    }


    MPI_Finalize();

    return 0;

}
```

```
arpita@DESKTOP-9FJDKRQ:~$ nano mp3.c
arpita@DESKTOP-9FJDKRQ:~$ mpicc mp3.c -o mp3
arpita@DESKTOP-9FJDKRQ:~$ mpirun -np 1 ./mp3
Rank 0 -> start: 0 end: 9999
Array Size: 10000 | Processes: 1 | Execution Time: 0.000027 seconds

Rank 0 -> start: 0 end: 99999
Array Size: 100000 | Processes: 1 | Execution Time: 0.000158 seconds

Rank 0 -> start: 0 end: 999999
Array Size: 1000000 | Processes: 1 | Execution Time: 0.001405 seconds

arpita@DESKTOP-9FJDKRQ:~$ mpirun -np 2 ./mp3
Rank 0 -> start: 0 end: 4999
Rank 1 -> start: 5000 end: 9999
Array Size: 10000 | Processes: 2 | Execution Time: 0.000007 seconds

Rank 0 -> start: 0 end: 49999
Rank 1 -> start: 50000 end: 99999
Array Size: 100000 | Processes: 2 | Execution Time: 0.000065 seconds

Rank 1 -> start: 500000 end: 999999
Rank 0 -> start: 0 end: 499999
Array Size: 1000000 | Processes: 2 | Execution Time: 0.000715 seconds
```

```
arpita@DESKTOP-9FJDKRQ:~$ mpirun -np 3 ./mp3
Rank 1 -> start: 3334 end: 6666
Rank 0 -> start: 0 end: 3333
Rank 2 -> start: 6667 end: 9999
Array Size: 10000 | Processes: 3 | Execution Time: 0.000005 seconds

Rank 2 -> start: 66667 end: 99999
Rank 0 -> start: 0 end: 33333
Rank 1 -> start: 33334 end: 66666
Array Size: 100000 | Processes: 3 | Execution Time: 0.000111 seconds

Rank 2 -> start: 666667 end: 999999
Rank 0 -> start: 0 end: 333333
Rank 1 -> start: 333334 end: 666666
Array Size: 1000000 | Processes: 3 | Execution Time: 0.000541 seconds
```

```
arpita@DESKTOP-9FJDKRQ:~$ mpirun -np 4 ./mp3
Rank 2 -> start: 5000 end: 7499
Rank 0 -> start: 0 end: 2499
Rank 1 -> start: 2500 end: 4999
Rank 3 -> start: 7500 end: 9999
Array Size: 10000 | Processes: 4 | Execution Time: 0.000007 seconds

Rank 0 -> start: 0 end: 24999
Rank 3 -> start: 75000 end: 99999
Rank 1 -> start: 25000 end: 49999
Rank 2 -> start: 50000 end: 74999
Array Size: 100000 | Processes: 4 | Execution Time: 0.000200 seconds

Rank 0 -> start: 0 end: 249999
Rank 2 -> start: 500000 end: 749999
Rank 3 -> start: 750000 end: 999999
Rank 1 -> start: 250000 end: 499999
Array Size: 1000000 | Processes: 4 | Execution Time: 0.000481 seconds
```